Advanced Programming

Stream API

The Context

- We have **data** stored in various collections.
- We must perform various operations on that data: *sorting, filtering, grouping, mapping, etc.*
- Using only iterators, we may repeat the same constructions over and over again.
- We need a mechanism that is: *terser (concise, declarative), functional, less mutable, loose-coupled, more performant(?)* when working with collections.

Example

```
int sum = 0;
for(Item e : items) {
    if (e.getWeight() <= 10) {
        continue;
    }
    if (!e.isAvailable()) {
        continue;
    }
    sum += e.getPrice();
}
```

What is a *stream*?

- *A stream* is a sequence of elements supporting sequential and parallel aggregate operations.
- Elements can be *objects* or *primitive* values (int, long, double). See *java.util.stream package*.
- Example

```
int sum = items.stream()
               .filter(e -> e.getWeight() > 10)
               .filter(e -> e.isAvailable())
               .mapToInt(e -> e.getPrice())
               .sum();
```

Stream Pipelines

- To perform a computation, stream operations are composed into a **stream pipeline (query).**
- A stream pipeline consists of a **source** (which might be an array, a collection, a generator function, an I/O channel, etc), zero or more **intermediate operations** (which transform a stream into another stream, such as *filter(Predicate)*), and a **terminal operation** (which produces a result or side-effect, such as *count()* or *forEach(Consumer)*).
- Streams are **lazy**; computation on the source data is only performed when the terminal operation is initiated, and source elements are consumed only as needed.
- **Collections** are concerned with the efficient management of and their elements, **streams** are concerned with declaratively describing their source and the computational operations which will be performed.

Functional Interfaces

- A functional interface is any interface that contains only one abstract method.
- Instead of using an anonymous class, you use a *lambda expression*, omitting the name of the interface and the name of the method.

```
Arrays.sort(p, (Person p1, Person p2) -> {
    return p1.getName().compareTo(p2.getName());
});
```

```
@FunctionalInterface
public interface Comparator { ...}
```

java.util.function packages

Functional interfaces, having various methods:

- **Function**
  - *apply*: accepts one argument and produces a result
- **Predicate**
  - *test*: boolean-valued function of one argument
- **Consumer**
  - *accept*: a single input argument and returns no result
- **Supplier**
  - *get:* does not take any argument and produces a value
- ...

Instances of these types will be usually created
using **lambda-expressions** and **method references**.

Under the Hood

```
int sum = Arrays.asList(1, 2, 3, 4, 5).stream()
                .filter(x -> x >= 3)
                .mapToInt(Integer::intValue)
                .sum();
```

- *filter*

```
Stream<T> filter(Predicate<? super T> predicate);
```

- *mapToInt*

```
IntStream mapToInt(ToIntFunction<? super T> mapper);
```

- *sum*

```
int sum();
```

Explicit Use of Functional Interfaces

- *Predicate*

```
Predicate<Integer> pred1 = x -> (x > 2);
Predicate<Integer> pred2 = x -> (x < 5);
System.out.println(pred1.and(pred2).negate().test(1));
```

- *Function*

```
Function<String, Integer> fun1 = str -> str.length();
Function<Integer, Integer> fun2 = x -> x*x;
System.out.println(fun1.andThen(fun2).apply("Hello"));
```

- **...**

Stream Creation

- *Stream.empty*
- *Collection.stream*

```
Stream<String> s = collection.stream();
```

- *Stream.of*

```
Stream<String> s = Stream.of("a", "b", "c");
```

- *Arrays.stream*

```
int[] arr = {1,2,3};
IntStream s = Arrays.stream(arr);
```

- *range, rangeClosed*

```
LongStream longStream = LongStream.range(0, 10);
```

- *...*

Intermediate Operations

filter, map, flatMap, distinct, sorted, limit, skip, etc.

- Intermediate operations are **applied on a source** stream.
- Intermediate operations return **a new modified stream**.

```
Stream<String> s = Stream.of("a", "b").skip(1);
```

- Operations can be **chained**

```
stream.sorted().distinct().skip(1);
```

- Intermediate operations are ended by a **single terminal operation**:

```
long x = Stream.of("a", "a").distinct().count();
```

Terminal Operations

sum, count, min, max, average, reduce, forEach, collect, etc.

- Are **applied on a (intermediate)** stream.
- Terminal operations return **a value** (not a stream).

```
Stream.of("a", "b").count();

Stream.of("Hello", "World").forEach(System.out::println);

persons.stream()
    .filter(p -> p.getAge() >= 18)
    .mapToInt(Person::getAge)
    .average()
    .getAsDouble();
```

- Terminal operations cannot be **chained**

Iterating, Filtering and Matching

stream → filter → findFirst, findAny, allMatch, anyMatch, noneMatch, etc.

- Substitute for: **for-each** and **while** loops.
- “Classical” iteration

```
for (Person p : persons) {
    if (p.getAge() >= 18 && p.getName().endsWith("escu")) {
        return p;
    }
}
return null;
```

- “Fancy” stream-based iteration

```
persons.stream()
    .filter(p -> p.getAge() >= 18)
    .filter(p -> p.getName().endsWith("escu"))
    .findFirst()
    .orElse(null);
```

Optional<Person>

Optional

Tired of Null Pointer Exceptions?

- A *container (wrapper)* object which may or may not contain a non-null value. If a value is present, *isPresent()* will return true and *get()* will return the value.
- What problem is it trying to solve?

```
String countryName =
person.getAddress().getCountry().getName();
```

- Important for stream intermediate operations.
- Groovy has a *safe navigation operator:*

```
person?.getAddress()?.getCountry()?.getName();
```

- Kotlin has a type system that distinguishes between *references that can hold null and those that can not*.

```
var a: String = "abc"
a = null //compilation error
```

```
var b: String? = "abc"
b = null // ok
```

Using Optional

- Explicitly

```
String name = "John"; //may also be null
Optional<String> opt = Optional.ofNullable(name);

assert opt.isPresent(); assert !opt.isEmpty();
assert opt.get().equals(name);

opt.ifPresent(System.out::println);
int len = opt.orElse("").length();
```

- Cascading operations

```
String countryName = person.map(Person::getAddress)
            .map(Address::getCountry)
            .map(Country::getName)
            .orElse("UNKNOWN");
```

Reducing

- Reducing a sequence of elements to some value according to a specified function.
- The method **reduce** takes two parameters: a start value, and an accumulator function.

```
List<Integer> numbers = Arrays.asList(1, 2, 3);

Integer reduced = numbers
    .stream()
    .reduce(0, (a, b) -> a + b);

//same as
Integer reduced = numbers.stream()
    .reduce(0, Integer::sum);
```

Mapping

- Converting elements of a *Stream*, by applying a special function to them, and collecting the new elements into another *Stream.*

```
items.stream()
    .map(item -> item.getProduct())
    .mapToDouble(product -> product.getPrice())
    .average().getAsDouble());
```

- The *flatMap* returns a stream consisting of the results of replacing each element of this stream with the contents of a mapped stream produced by applying the provided mapping function to each element.

```
orders.flatMap(order -> order.getItems().stream())
   .forEach(System.out::println);
```

Collecting

- Converting a stream to a *Collection* or a *Map* .
- The utility class *Collectors* provides a solution for almost all typical collecting operations.
- Collecting to a *List, Set,* etc.

```
List<String> names = persons.stream()
    .map(p -> p.getName())
    .collect(Collectors.toList());
```

- Collecting to a *String*

```
String listToString persons.stream()
      .map(Person::getName)
      .collect(Collectors.joining(", ", "[", "]"));
```

Sorting

- Using the natural order

```
List<String> sortedList = list.stream()
   .sorted().collect(Collectors.toList());
```

- Using a *Comparator*

```
List<Person> sortedList = persons.stream()
   .sorted(Comparator.comparingInt(User::getAge))
   .collect(Collectors.toList());
```

- Reversed

```
List<String> reversedList = list.stream()
   .sorted().reversed().collect(Collectors.toList());
```

- ...

Grouping

```
Team barca = new Team("FC Barcelona");
Team real = new Team("Real Madrid");
Player[] players = {
    new Player("Lionel Messi", barca),
    new Player("Louis Suarez", barca),
    new Player("Antoine Griezman", barca),
    new Player("Karim Benzema", real),
    new Player("Eden Hazard", real)
};

Map<Team, List<Player>> teamPlayers =
  Arrays.stream(players)
        .collect(Collectors.groupingBy(Player::getTeam));

System.out.println(teamPlayers);

// {Real Madrid=[Karim Benzema, Eden Hazard],
// FC Barcelona=[Lionel Messi, Louis Suarez, Antoine Griezman]}
```

a classifier *function* mapping input elements to keys

Counting and Summing

- Counting

```
Map<String, Long> counting = items.stream()
    .collect(Collectors.groupingBy(
                    Item::getName,
                    Collectors.counting()));
```

a classifier *function* mapping input elements to keys

a *collector* implementing the downstream reduction

- Summing

```
Map<String, Integer> sum = items.stream()
    .collect(Collectors.groupingBy(
                    Item::getName,
                    Collectors.summingInt(Item::getQty)));
```

- ...

Parallel Streams

- Stream pipelines may execute either **sequentially (default)** or in **parallel**.
- `stream.parallel()` creates a parallel stream.
- Advantage: it uses a **multi-threaded approach** in order to execute the functions inside the pipeline, making better use of multi-core processors.
- Disadvantage: it has a much higher overhead compared to a sequential one. It pays off only if there is a large number of independent tasks.

Using a *parallelStream*

```
//Sequential
IntStream stream1 = IntStream.rangeClosed(1, 10);
stream1.forEach(System.out::println);
// 1 2 3 4 5 6 7 8 9 10

//Parallel
IntStream stream2 = IntStream.rangeClosed(1, 10);
stream2.parallel().forEach(System.out::println);
// 7 6 3 2 5 9 1 10 4 8
```

Improving Perfomance

```
private boolean isPrime(long number) {
    long d = 2;
    while (d * d <= number) {
        if (number % (d++) == 0) return false;
    }
    return true;
}

private void test() {
    int n = 100_000_000; //Looking for a prime number in a large array
    long arr[] = new long[n];
    for (int i = 0; i < n - 1; i++) {
        arr[i] = 100003 * 100003;
    }
    arr[n - 1] = 2;

    long nr1 = Arrays.stream(arr)
       .filter(this::isPrime).findAny().getAsLong();

    long nr2 = Arrays.stream(arr).parallel()
       .filter(this::isPrime).findAny().getAsLong();
    }
}
```

7.435 s

2.677 s

Recap

- *Stream* - sequence of elements supporting sequential and parallel aggregate operations.
- *Pipeline* - a sequence of aggregate operations.

```
persons.stream()
    .filter(p -> p.getAge() >= 18)
    .filter(p -> p.getName().endsWith("escu"))
    .forEach(s -> System.out.println(s.getName()));
```

- *Mapping* and Terminal ops.

```
double averageAge = persons.stream()
    .filter(p -> p.getAge() >= 18)
    .mapToInt(Person::getAge)
    .average()
    .getAsDouble();
```

- **Source**
  array, collection, ...
- **Intermediate** operations
  filter, distinct, sorted,...
- **Mapping** operations
  map, mapToInt, ...
- **Terminal** operations
  average, min, max,...